Campus Recruitment Training — Phase 2

Post-Assessment — Complete Solutions

Duration: 90 Minutes | Total Marks: 100

> **Instructions:** All questions are code-based. Write clean, readable Java/YAML/JS code as applicable. Attempt all sections. Partial credit is awarded where marked. You may write short comments to explain your logic — it helps in evaluation.

Section A — Spring Boot & Request Lifecycle [25 marks]

Q1. Predict the output when the Spring Boot application starts. Write output in order with one-line reason for each. **[4M]**

Answer:

Output (in order):

```
Constructor called
Init called
```

Reason for each line:

- "Constructor called" — Spring creates the bean by calling its constructor first, as part of the bean instantiation phase.
- "Init called" — After the bean is fully instantiated and all dependencies are injected, Spring calls the method annotated with @PostConstruct. This is the initialization callback that runs after construction.

Q2. The endpoint always returns an empty response body. Identify the mistake and write the corrected line. **[3M]**

Answer:

Bug: The ResponseEntity is constructed without passing the User object as the body.

Buggy line:

```
return new ResponseEntity<>(HttpStatus.OK);
```

Corrected line:

```
return new ResponseEntity<>(u, HttpStatus.OK);
```

Reason: ResponseEntity<>(HttpStatus.OK) only sets the status code. The body is null. The correct constructor is ResponseEntity<>(body, status) which puts the User object into the response body.

Q3. Write POST /api/students with validation — DTO and controller. **[5M]**

See code file: Q3_StudentController.java (submitted separately)

Q4. What annotation should be added to the transfer() method and why? What happens if missing? **[3M]**

Answer:

Annotation to add:

```
@Transactional
```

Why it is needed:

The transfer() method performs two related operations — debiting one account and crediting another — that must either both succeed or both fail together. @Transactional wraps both repo.save() calls inside a single database transaction.

What happens if it is missing:

If the first repo.save(from) succeeds but the second repo.save(to) fails (e.g., due to a database error or runtime exception), the debit is permanently committed to the database while the credit is rolled back. This leaves the database in an inconsistent state — money vanishes from the sender but never reaches the receiver.

Q5. Write JPA Employee and Department entity classes. **[5M]**

See code file: Q5_JPA_Entities.java (submitted separately)

Q6. JWT Filter: (a) Explain what it does. (b) Find and fix the bug. **[5M]**

Answer:

(a) What the filter does:

This is a JWT authentication filter. It intercepts every incoming HTTP request, reads the Authorization header, and extracts the Bearer token. It then uses JwtUtil to extract the username from the token. If a valid username is found, it creates an authentication object and stores it in the Spring Security context so the rest of the application treats the request as authenticated.

(b) The bug:

The token is extracted using substring(6) but it should be substring(7).

The Authorization header format is: Bearer <token>

"Bearer " is 7 characters (B-e-a-r-e-r-SPACE). Using substring(6) keeps the leading space, making the token " eyJh..." (with a space prefix), which makes all JWT validation fail.

Buggy line:

```
String token = header.substring(6);   // WRONG — includes the space
```

Corrected line:

```
String token = header.substring(7);   // CORRECT — skips "Bearer "
```

Section B — JPA, SQL & Data Layer [20 marks]

Q7. Write a @Query to find Products by price and category using named parameters. [4M]

See code file: Q7_Q9_Repository.java (submitted separately)

Q8. Explain the N+1 query problem with an example, its cause, and a fix using JOIN FETCH. [4M]

Answer:

What is N+1:

When you load a list of N entities and then access a lazy-loaded association on each one, JPA fires one query to get the list plus N more queries (one per entity) to load each association. For 10 orders, this means 11 queries instead of 1.

Example:

```
List<Order> orders = orderRepo.findAll();  // Query 1
for (Order o : orders) {
    o.getItems();  // Triggers 1 query per order — N more queries
}
```

Most common cause:

Lazy loading (@OneToMany or @ManyToOne with fetch = LAZY) on an association that is accessed inside a loop. Each get triggers a separate SQL SELECT.

Fix using JOIN FETCH:

```
@Query("SELECT o FROM Order o JOIN FETCH o.items")
List<Order> findAllWithItems();
```

JOIN FETCH tells JPA to load the order and all its items together in a single SQL JOIN query.

Q9. Second highest salary: SQL subquery + JPQL @Query method. [4M]

See code file: Q7_Q9_Repository.java (submitted separately)

SQL logic: SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees)

Q10. Four Spring Data JPA derived query method signatures for Student entity. [5M]

See code file: Q10_StudentRepository.java (submitted separately)

Q11. 11 SQL queries instead of 1 — what is this problem? Identify the cause and write the fix. [3M]

Answer:

Problem name:

This is the N+1 Query Problem.

Line that causes the problem:

```
System.out.println(o.getItems().size());
```

Calling o.getItems() on a lazily loaded @OneToMany collection triggers a separate SQL query for each Order to fetch its items. With 10 orders, that is 1 query (findAll) + 10 queries (one per order) = 11 total.

One-line fix:

```
@Query("SELECT o FROM Order o JOIN FETCH o.items")
```

Use JOIN FETCH in the repository query to eagerly load items together with orders in a single SQL JOIN, eliminating all the extra queries.

Section C — Microservices & System Design [25 marks]

Q12. Eureka registration config (application.yml) and main class annotation for product-service. **[4M]**

See code files: Q12_application.yml and Q12_ProductServiceApplication.java (submitted separately)

Q13. What is a Circuit Breaker? Why is it needed? Write a method with @CircuitBreaker and fallback. **[5M]**

Answer:

What is a Circuit Breaker:

A Circuit Breaker is a design pattern for microservices that prevents cascading failures. Like an electrical circuit breaker, it monitors calls to an external service. When the failure rate crosses a threshold, the circuit "opens" — further calls are immediately rejected and the fallback is returned instead of attempting a call that will likely fail.

Why it is needed:

In a microservices architecture, one slow or down service can cause all services that call it to hang, exhaust thread pools, and eventually crash. The Circuit Breaker fails fast and gives a default response, keeping the caller healthy while the downstream service recovers.

States of a Circuit Breaker:

- CLOSED — Normal operation. Calls go through. Failures are counted.
- OPEN — Failure threshold exceeded. Calls are blocked; fallback is returned immediately.
- HALF-OPEN — After a wait period, a few test calls are allowed. If they succeed, the circuit closes again.

Code: See Q13_CircuitBreaker.java (submitted separately)

Q14. What is an API Gateway? 2 benefits? Write Spring Cloud Gateway routes config. **[6M]**

Answer:

What is an API Gateway:

An API Gateway is a single entry point for all client requests in a microservices architecture. It acts as a reverse proxy — routing requests to the appropriate downstream microservice based on the URL path, and optionally applying cross-cutting concerns like authentication, rate limiting, and logging.

Two benefits:

- Single Entry Point / Simplified Client: The frontend only needs one URL (e.g., http://gateway:8080) instead of knowing the address of every microservice. Routing rules in the gateway decide which service handles each request.

- Centralized Cross-Cutting Concerns: Authentication (JWT validation), rate limiting, logging, and SSL termination can all be handled once at the gateway instead of duplicating this logic in every microservice.

Code: See Q14_gateway_application.yml (submitted separately)

Q15. What is Service Discovery and why is it needed? Write @LoadBalanced RestTemplate bean. **[4M]**

Answer:

What is Service Discovery:

Service Discovery is the mechanism by which microservices automatically locate each other without hardcoding IP addresses or ports. In a cloud or containerized environment, services are scaled up/down and their IPs change constantly. A service registry (like Eureka) acts as a phone book — each service registers its current address, and callers look up the address by service name at runtime.

Why not hardcode IPs:

- Services are ephemeral in containers — their IPs change on restart or scaling.
- Multiple instances may run simultaneously (load balancing across them).
- Hardcoded IPs require redeployment every time infrastructure changes.

Tool used in class: Spring Cloud Netflix Eureka

Code: See Q15_RestTemplateConfig.java (submitted separately)

Q16. Monolith split: 3 benefits, 2 challenges, and how Eureka + Gateway work together for a React frontend. **[6M]**

Answer:

(a) 3 Benefits of splitting into microservices:

- Independent Deployment: Each service (UserService, OrderService, ProductService) can be deployed, updated, or rolled back independently without touching the others. A bug in ProductService does not require redeploying UserService.
- Independent Scalability: If OrderService gets high traffic, only that service can be scaled horizontally (more instances). In a monolith, you must scale the entire app even if only one module needs more resources.
- Team Autonomy & Technology Flexibility: Different teams can own different services, work independently, and even choose different tech stacks for each service.

(b) 2 Challenges:

- Distributed Data Management: In a monolith, all modules share one database. In microservices each service has its own DB, making cross-service queries and maintaining data consistency harder (requires patterns like Saga or eventual consistency).

- Increased Operational Complexity: Instead of one app to deploy and monitor, you now manage multiple services, service discovery, inter-service communication, distributed tracing, and container orchestration.

(c) How Eureka + API Gateway work together for a React frontend:

1. Each microservice (UserService, OrderService, ProductService) starts up and registers itself with the Eureka server, providing its service name and current IP:port.

2. The API Gateway also registers with Eureka and is configured with routing rules: requests to /api/users/** go to user-service, /api/orders/** go to order-service, etc.

3. The React frontend always sends requests to the single gateway URL (e.g., http://gateway:8080). It never knows about individual service addresses.

4. The gateway looks up the target service in Eureka using its name, selects an available instance (load balancing), and forwards the request.

5. The response flows back through the gateway to the React app. From the frontend's perspective, it is just one unified API.

Section D — Testing & Docker [20 marks]

Q17. Write JUnit 5 + Mockito unit test for ProductService.getById(). [5M]
See code file: Q17_ProductServiceTest.java (submitted separately)

Q18. Difference between @Mock (Mockito) and @MockBean (Spring Boot Test). Example of @MockBean with @SpringBootTest. [4M]

Answer:

	@Mock (Mockito)	@MockBean (Spring Boot)
Spring Context	Does NOT start Spring context	Starts full Spring context (slow)
Use when	Unit testing a class in isolation (no Spring needed)	Integration testing controllers/services inside Spring context
Extension needed	`@ExtendWith(MockitoExtension.class)`	`@SpringBootTest`
Speed	Fast (milliseconds)	Slow (full app startup)

Example using @MockBean with @SpringBootTest:

```
@SpringBootTest
class UserControllerIntegrationTest {

    @MockBean
    private UserService userService;  // Replaces real bean in Spring context

    @Autowired
    private MockMvc mockMvc;

    @Test
    void testGetUser() throws Exception {
        when(userService.findById(1L)).thenReturn(new User(1L, "Alice"));
        mockMvc.perform(get("/api/users/1"))
               .andExpect(status().isOk());
    }
}
```

Q19. Write a Dockerfile for a Spring Boot app + docker build and run commands. [5M]
See code file: Q19_Dockerfile (submitted separately)

Q20. Write docker-compose.yml with db (postgres:15) and app (myapp:latest) services. **[6M]**

See code file: Q20_docker-compose.yml (submitted separately)

Section E — Frontend Integration (Bonus) [10 marks]

Q21. Write async function loadUsers() using fetch API with success/error handling. **[5M]**

See code file: Q21_loadUsers.js (submitted separately)

Q22. React UserList infinite loop — identify the bug, fix it, and add loading state. **[5M]**

Answer:

(a) What is wrong — the bug:

```
useEffect(() => {
    fetch('/api/users')...
});   // <-- NO dependency array!
```

The useEffect has no second argument (no dependency array). Without it, React runs the effect after every render. The fetch inside calls setUsers(data), which updates state and triggers a re-render, which runs the effect again, which calls setUsers again — creating an infinite render/fetch loop.

(b) The fix:

```
useEffect(() => {
    fetch('/api/users')...
}, []);  // Empty [] = run only once on mount
```

Adding an empty dependency array [] as the second argument tells React to run this effect only once — immediately after the first render (component mount). It will never run again unless the component unmounts and remounts.

(c) Loading state added:

See code file: Q22_UserList.jsx (submitted separately)

The loading state is initialised to true. When the fetch completes (or fails), setLoading(false) is called. While loading is true, the component returns Loading... instead of the user list.